Същност и обхват

Софтуерното инженерство е инженерна дисциплина, която прилага систематични, дисциплинирани и количествено измерими подходи към проектирането, разработката, поддръжката и управлението на софтуерни системи с високо качество. За разлика от индивидуалното програмиране, то обхваща целия жизнен цикъл на продукта — от първоначалната идея до извеждането му от употреба.

Софтуерът не е само програмен код. Той включва три основни компонента:

- **Инструкции:** Компютърни програми, които осигуряват желаната функционалност и производителност.
- **Структури от данни:** Позволяват на програмите да съхраняват и обработват информация ефективно.
- **Документация:** Описва работата и използването на програмите.

Ключови характеристики:

- Софтуерът се **разработва**, а не се произвежда в класическия смисъл.
- Той не се износва физически, но подлежи на **остаряване** поради технологични иновации и промени в бизнес нуждите.

Видове софтуерни продукти и системи

Софтуерните продукти се делят на две основни категории според техния целеви пазар:

1. **Общи (Generic):** Самостоятелни системи, предназначени за широкия пазар (напр. текстови редактори).
2. **Поръчани (Customized):** Разработени специално за нуждите на конкретен клиент.

Вид софтуер	Описание	Примери	Вид софтуер	Описание	Примери
Системен	Обслужва други програми; интензивна комуникация с хардуера.	ОС, драйвери, компилатори.	**Уеб приложения**	Многослойни системи, достъпни през браузър.	Социални мрежи, онлайн магазини.
Приложен	Решава специфични нужди на потребителите.	Текстови редактори, плейъри.	**Изкуствен интелект**	Използва нечислови алгоритми за сложни проблеми.	Разпознаване на образи, невронни мрежи.
Научен	Използва се за симулации, анализи и емпирично тестване.	Симулатори на климата, CAD системи.	**Наследен (Legacy)**	Стари системи с критична важност за бизнеса.	Банкови системи, разработени преди десетилетия.
Вграден (Embedded)	Интегриран в хардуерни устройства.	Софтуер в перални, автомобили, климатици.			

Софтуерният процес е последователност от предвидими стъпки и съвкупност от процедури, организирани за създаването на висококачествен продукт.

Основни фази на разработка

1. **Изисквания:** Събиране и анализ на нуждите на потребителите.
2. **Проектиране:** Архитектурно и детайлно планиране на системата.
3. **Реализация:** Самото програмиране (кодиране).
4. **Тестване:** Проверка на компонентите (Unit testing), интеграцията и цялата система.
5. **Внедряване:** Инсталиране и предоставяне на клиента.
6. **Поддръжка:** Корекции и актуализации след внедряването.

Универсални дейности в процеса

- **Комуникация:** Сътрудничество с клиента за изясняване на функционалността.
- **Планиране:** Дефиниране на ресурси, рискове и график.
- **Моделиране:** Създаване на технически спецификации чрез анализ и дизайн.
- **Конструиране:** Генериране на код и тестване.
- **Внедряване:** Оценка на продукта от крайния потребител.

Езици и шаблони за моделиране

За визуализация на системите най-често се използва **UML (Unified Modeling Language)**. Моделите могат да бъдат:

- **Описателни:** Как се разработва системата.
- **Предписателни:** Как би трябвало да се разработва.

Шаблонът на процес е описание на доказано решение на общ проблем. Всеки шаблон съдържа: контекст, проблем, решение, краен контекст и свързани шаблони.

Модел	Характеристика	Предимства	Недостатъци	Подходящ за...
Водопад (Waterfall)	Линеен и последователен подход.	Яснота, лесно управление, строга документация.	Негъвкав; късно откриване на грешки; изисква пълни изисквания в началото.	Проекти със стабилни и добре известни изисквания.
RAD (Бърза разработка)	Високоскоростен, модулен, паралелен (60-90 дни).	Бърз резултат чрез готови компоненти и автоматичен код.	Не е за сложни архитектури; изисква модулност.	Малки до средни проекти с ясни модули.
Прототипен	Създаване на ранна версия (еволюционна или изхвърляема).	Помага за изясняване на изискванията; ранна обратна връзка.	Може да изразходва много ресурси; риск от лоша архитектура.	Проекти с неясни потребителски изисквания.
Спираловиден	Итеративен модел с фокус върху **анализа на риска**.	Гъвкав; подходящ за промени; систематично управление на риска.	Скъп; сложен за управление; изисква експерти по риска.	Големи, сложни и високорискови проекти.
Постъпков (Incremental)	Системата се изгражда и доставя на части (инкременти).	Ранна доставка на ядрото; по-лесна адаптация.	Изисква много добра първоначална архитектура.	Системи, които трябва бързо да пуснат базова функция.

Agile методите поставят акцент върху хората, работещия софтуер и адаптацията към промени, вместо върху строгото следване на планове и документация.

Scrum

Scrum организира работата в кратки итерации, наречени **спринтове** (1-4 седмици).

- **Роли:** Product Owner (представител на бизнеса), Scrum Master (фасилитатор) и Development Team (самоорганизиращ се екип).
- **Артефакти:** Product Backlog (списък с всички нужди) и Sprint Backlog (задачи за текущия спринт).
- **Събития:** Планиране, Ежедневна среща (Daily Scrum), Преглед на спринта (Review) и Ретроспектива.

Extreme Programming (XP)

XP се фокусира върху техническото качество чрез специфични практики:

- **Чифтно програмиране:** Двама души работят на един компютър.
- **TDD (Test-Driven Development):** Първо се пише тест, после минималният код.
- **Непрекъсната интеграция:** Често обединяване на програмния код.
- **Рефакториране:** Подобряване на вътрешната структура на кода без промяна на поведението му.

+ Концепция за многократна употреба (Reusability)

Практиката за използване на съществуващи компоненти, библиотеки и рамки (frameworks) за пестене на време и повишаване на качеството.

- **COTS (Commercial Off-The-Shelf):** Готови софтуерни продукти, които предлагат API (напр. платежни системи като Stripe).
- **Компонентно-базирана разработка:** Сглобяване на системи от независими единици с дефинирани интерфейси (**Requires** - какво му трябва, **Provides** - какво дава).
- **Шаблони за проектиране (Design Patterns):** Абстрактни решения на архитектурни проблеми (Създаващи, Структурни, Поведенчески).